MOHAN BABU UNIVERSITY

Anveshana Club

Backend Schema & API Specification

DATABASE: MONGODB ATLAS
ARCHITECTURE: NODE.JS / EXPRESS REST API
ALIGNED WITH PRD & TRD SPECIFICATIONS

1. Database Schema Definitions (MongoDB)

All collections utilize `ObjectId` for primary keys (`_id`) and include Mongoose automatic `createdAt` and `updatedAt` timestamps.

1.1 Administrators

Purpose: Secure storage for administrative accounts. Public registration is disabled.

Field	BSON Type	Required	Default	Validation / Notes
_id	`ObjectId`	Yes	Auto	Primary Key.
email	`String`	Yes	-	Valid email format; lowercase. Unique.
passwordHash	`String`	Yes	-	Bcrypt hash. Never returned in API.
role	`String`	Yes	`admin`	Enum: `['admin', 'superadmin']`.
isActive	`Boolean`	Yes	`true`	Enables immediate account suspension.
lastLogin	`Date`	No	null	Updated upon successful authentication.

1.2 Events

Purpose: Core entity storing event metadata, publication status, and external registration links.

Field	BSON Type	Required	Default	Validation / Notes
_id	`ObjectId`	Yes	Auto	Primary Key.
title	`String`	Yes	-	Max 100 characters.
slug	`String`	Yes	-	URL-safe string. Must be unique.
shortDescription	`String`	Yes	-	Max 200 characters.
fullDescription	`String`	Yes	-	Rich text / HTML string. Sanitized on input.
bannerMedia	`Object`	Yes	-	`{ publicId: String, url: String }` (Cloudinary).
eventDate	`Date`	Yes	-	ISO 8601 Date. Used for chronological sorting.
time	`String`	Yes	-	E.g., "10:00 AM IST".
venue	`String`	Yes	-	Location string.
eligibility	`String`	No	null	E.g., "All SoC Students".
registrationDeadline	`Date`	No	null	Auto-closes registration UI if passed.
registrationUrl	`String`	No	null	Valid HTTPS Google Forms URL.
status	`String`	Yes	`upcoming`	Enum: `upcoming, open, closed, completed`.
isPublished	`Boolean`	Yes	`false`	Controls public visibility.

1.4 TeamCategories

Purpose: Logical grouping for the core team structure.

Field	BSON Type	Required	Default	Validation / Notes
_id	`ObjectId`	Yes	Auto	Primary Key.
name	`String`	Yes	-	E.g., "Technical Team".
slug	`String`	Yes	-	URL-safe string. Must be unique.
description	`String`	No	null	Short summary of the team's role.
displayOrder	`Number`	Yes	`0`	Determines UI rendering priority.
isActive	`Boolean`	Yes	`true`	Hide/show category globally.

1.5 TeamMembers

Purpose: Individual member profiles.

Field	BSON Type	Required	Default	Validation / Notes
categoryId	`ObjectId`	Yes	-	Foreign Key referencing `TeamCategories`.
name	`String`	Yes	-	Full name.
designation	`String`	Yes	-	E.g., "Lead Developer".
biography	`String`	No	null	Max 300 characters.
profileMedia	`Object`	No	null	`{ publicId, url }`. Fallback UI handled by frontend if null.
linkedinUrl	`String`	No	null	Must be a valid HTTPS URL.
githubUrl	`String`	No	null	Must be a valid HTTPS URL.
portfolioUrl	`String`	No	null	Must be a valid HTTPS URL.
displayOrder	`Number`	Yes	`0`	Used to sort leads above members.
isPublished	`Boolean`	Yes	`true`	Controls public visibility.

1.6 Feedback

Purpose: Collection of user ratings and event feedback.

Field	BSON Type	Required	Default	Validation / Notes
rating	`Number`	Yes	-	Min: 1, Max: 5.
feedbackText	`String`	No	null	Max 1000 characters. XSS sanitized.
status	`String`	Yes	`pending`	Enum: `pending, reviewed, archived`.
ipHash	`String`	Yes	-	One-way hash of IP address for rate-limiting.

1.7 ClubSettings (Singleton)

2. Relationships & Data Lifecycle

- **Events ↔ Galleries (1:1):** Enforced via `eventId` unique index on the Galleries collection. If an event is deleted via the Admin dashboard, Mongoose pre-remove hooks MUST delete the associated `EventGalleries` document AND trigger a Cloudinary API call to destroy all associated `publicId` image assets to prevent orphaned binaries.
- **Categories ↔ Members (1:N):** Deleting a `TeamCategory` is restricted if active `TeamMembers` reference it. The API returns a 409 Conflict. Members must be reassigned or deleted first.
- **Media Replacement:** When updating an event banner or member profile photo, the API must delete the old Cloudinary asset before saving the new reference to MongoDB.
- **Archival vs. Deletion:** Standard practice for completed events is to update `status: 'archived'` or `isPublished: false` rather than hard deletion, preserving historical club data.

3. API Specification (Public)

All API responses follow a standard envelope: `{ "success": Boolean, "data": Object|Array, "error": String|Null, "meta": Object }`. Public routes are unauthenticated.

Method	Endpoint Path	Purpose	Query / Body
`GET`	`/api/v1/public/settings`	Retrieve club info & contact.	None
`GET`	`/api/v1/public/events`	List published events. Native Mongoose filter: `isPublished: true`.	`?status=upcoming` `?limit=10&page=1`
`GET`	`/api/v1/public/events/:slug`	Retrieve single event details.	None
`GET`	`/api/v1/public/events/:slug/gallery`	Retrieve event image array.	None
`GET`	`/api/v1/public/team`	Get active categories & members.	None
`POST`	`/api/v1/public/feedback`	Submit rating/feedback.	Body: `{ rating, feedbackText }`

4. API Specification (Administrative)

Protected by `isAuth` middleware verifying a secure JWT cookie. API returns `401 Unauthorized` if invalid.

Auth & Session Endpoints

Method	Endpoint Path	Purpose	Payload
`POST`	`/api/v1/admin/auth/login`	Authenticate admin. Sets HttpOnly cookie.	`{ email, password }`
`POST`	`/api/v1/admin/auth/logout`	Destroy session (Clear cookie).	None
`GET`	`/api/v1/admin/auth/me`	Verify active token state for React Router.	Cookie Header

Content Management System (CMS) Endpoints

Method	Endpoint Path	Purpose	Behavior
`GET`	`/api/v1/admin/events`	List ALL events.	Includes drafts.
`POST`	`/api/v1/admin/events`	Create new event.	Returns new `_id`.
`PUT`	`/api/v1/admin/events/:id`	Update event metadata.	Can toggle `isPublished`.
`DELETE`	`/api/v1/admin/events/:id`	Delete event.	Triggers DB/Media cleanup.
`PUT`	`/api/v1/admin/events/:id/gallery`	Update event gallery.	Multipart form upload.
`POST`	`/api/v1/admin/team/categories`	Create category.	Validates slug uniqueness.
`PUT`	`/api/v1/admin/team/categories/:id`	Edit category.	Update name/order.
`POST`	`/api/v1/admin/team/members`	Create team member.	Validates Category exists.
`PUT`	`/api/v1/admin/team/members/:id`	Update team member.	Multipart form upload.
`GET`	`/api/v1/admin/feedback`	Review feedback list.	Supports pagination.
`PUT`	`/api/v1/admin/settings`	Update global settings.	Overwrites singleton.

5. Security and Authorization

- **Authentication:** Short-lived JWT stored strictly in an `HttpOnly`, `SameSite=Strict` cookie to mitigate XSS vector attacks. The token payload contains the `_id` and `role`.
- **Authorization:** The `requireAdmin` middleware intercepts all `/api/v1/admin/*` routes. It verifies the signature and checks the database to ensure the admin's `isActive` flag is still true, allowing instant revocation of compromised accounts.
- **Spam Prevention:** The public `POST /feedback` route utilizes `express-rate-limit` (max 3 submissions per IP per hour). Feedback text is stripped of HTML tags via `xss-clean` before database insertion.
- **Secret Management:** MongoDB URIs, JWT Secrets, and Cloudinary API Keys are injected via Node environment variables. They are never returned in JSON responses.

6. Cloudinary Integration Architecture

To secure media assets and optimize bandwidth, image handling is strictly proxied through the Express backend.

- **Upload Flow:** The React client sends the image via `multipart/form-data`. Express parses it using `multer` (validating mime-types: jpeg, png, webp) and enforces a 5MB size limit. The backend utilizes the Cloudinary Node SDK to stream the buffer directly to Cloudinary. Credentials remain hidden on the server.
- **Transformations:** Upload requests instruct Cloudinary to apply `f_auto,q_auto` (auto-format, auto-quality) and scale massive images to a maximum width of 1920px.
- **Error Handling:** If a MongoDB document fails validation *after* an image uploads, the `catch` block executes a `cloudinary.uploader.destroy(public_id)` command to delete the orphaned image and prevent free-tier storage exhaustion.

7. Indexes and Performance Strategy

MongoDB indexes are applied selectively to support the most frequent read queries generated by the public application flow.

- **Event Resolution:** A unique index on `Events.slug` ensures `GET /events/:slug` resolves in O(1) time.
- **Event Filtering:** A compound index on `Events.status_1_eventDate_-1` optimizes the homepage and listing pages where users request "Upcoming" events sorted chronologically.
- **Team Rendering:** An ascending index on `TeamCategories.displayOrder` guarantees deterministic UI rendering without expensive application-level sorting.
- **Authentication:** A unique index on `Administrators.email` prevents duplicate account creation and optimizes login lookups.